

Documentation_Terraform

Terraform

Terraform est un outil open-source d'Infrastructure as Code (IaC) développé par HashiCorp. Il permet de décrire, provisionner et gérer une infrastructure (machines virtuelles, réseaux, stockage...) sous forme de fichiers de configuration texte rédigés en HCL (HashiCorp Configuration Language). L'infrastructure devient ainsi reproductible et documentée. Dans ce projet nous allons utiliser Terraform pour automatiser le déploiement de machines virtuelles Debian 13 sous VMware Workstation Pro avant de construire un laboratoire composé de deux VMs.

Sommaire

- [1 - Préparation de l'environnement création de la VM template Debian 13](#)
 - [2 - Installation de Terraform et configuration du Provider VMware Workstation](#)
 - [3 - Premier déploiement de VM Debian 13](#)
 - [4 - Variables et gestion du cycle de vie](#)
 - [5 - Laboratoire de Cybersécurité déploiement de deux VMs](#)
 - [6 - Conclusion et compétences développées sur le projet](#)
-

1 - Préparation de l'environnement création de la VM template Debian 13

Afin d'utiliser terraform nous devons disposer d'une VM de référence (le "template") que Terraform pourra cloner pour créer de nouvelles machines. Cette VM Debian 13 doit être préparée sans interface graphique avec un serveur SSH pour permettre à Terraform de s'y connecter et avec Open-VM-Tools pour l'intégration VMware.

1.1 Installation de Debian 13 sans interface graphique

On installe Debian 13 dans VMware Workstation Pro en choisissant l'installation minimale, sans environnement de bureau.

`lsb_release - a`

```
test@TemplateTerraform:/$ lsb_release -a
No LSB modules are available.
Distributor ID: Debian
Description:    Debian GNU/Linux 13 (trixie)
Release:        13
Codename:       trixie
test@TemplateTerraform:/$
```

1.2 Installation du serveur SSH

On installe et active le serveur SSH sur la VM template pour permettre à Terraform de s'y connecter.

su -

nano /etc/apt/sources.list

```
GNU nano 8.4 /etc/apt/sources.list *
#deb cdrom:[Debian GNU/Linux 13.3.0 _Trixie_ - Official amd64 DVD Binary-1 with firmware 20260110-11:00]/ trixie contrib main non-free-firmware

deb http://deb.debian.org/debian/ trixie main non-free-firmware
deb-src http://deb.debian.org/debian/ trixie main non-free-firmware

deb http://security.debian.org/debian-security trixie-security main non-free-firmware
deb-src http://security.debian.org/debian-security trixie-security main non-free-firmware

# trixie-updates, to get updates before a point release is made;
# see https://www.debian.org/doc/manuals/debian-reference/ch02.en.html#_updates_and_backports
deb http://deb.debian.org/debian/ trixie-updates main non-free-firmware
deb-src http://deb.debian.org/debian/ trixie-updates main non-free-firmware
```

apt update && apt upgrade -y

```
root@TemplateTerraform:~# apt update && apt upgrade -y
Atteint : 1 http://deb.debian.org/debian trixie InRelease
Réception de : 2 http://security.debian.org/debian-security trixie-security InRelease [43,4 kB]
Atteint : 3 http://deb.debian.org/debian trixie-updates InRelease
43,4 ko réceptionnés en 0s (131 ko/s)
Tous les paquets sont à jour.
Sommaire :
  Mise à niveau de : 0. Installation de : 0Supprimé : 0. Non mis à jour : 0
root@TemplateTerraform:~# _
```

apt install -y openssh-server

systemctl enable ssh

systemctl status ssh

```

juil. 28 10:54:28 TemplateTerraform systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
juil. 28 10:54:28 TemplateTerraform sshd[1445]: Server listening on 0.0.0.0 port 22.
juil. 28 10:54:28 TemplateTerraform sshd[1445]: Server listening on :: port 22.
juil. 28 10:54:28 TemplateTerraform systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
root@TemplateTerraform:~# apt install -y openssh-server
openssh-server est déjà la version la plus récente (1:10.0p1-7+deb13u4).
Sommaire :
  Mise à niveau de : 0. Installation de : 0Supprimé : 0. Non mis à jour : 0
root@TemplateTerraform:~# systemctl enable ssh
Synchronizing state of ssh.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable ssh
root@TemplateTerraform:~# systemctl start ssh
root@TemplateTerraform:~# systemctl status ssh
• ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-07-28 10:54:28 CEST; 1min 49s ago
  Invocation: bed0db152cfc4079b8f1609a20f28005
     Docs: man:sshd(8)
           man:sshd_config(5)
   Main PID: 1445 (sshd)
      Tasks: 1 (limit: 2255)
     Memory: 1.3M (peak: 1.8M)
        CPU: 42ms
     CGroup: /system.slice/ssh.service
             └─1445 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

juil. 28 10:54:28 TemplateTerraform systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
juil. 28 10:54:28 TemplateTerraform sshd[1445]: Server listening on 0.0.0.0 port 22.
juil. 28 10:54:28 TemplateTerraform sshd[1445]: Server listening on :: port 22.
juil. 28 10:54:28 TemplateTerraform systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
root@TemplateTerraform:~#

```

1.3 Installation d'Open-VM-Tools

Open-VM-Tools est l'outil d'intégration entre la VM et VMware Workstation. Il permet notamment à l'hyperviseur de récupérer l'IP de la VM.

`apt install -y open-vm-tools`

```

root@TemplateTerraform:~# apt install -y open-vm-tools
open-vm-tools est déjà la version la plus récente (2:12.5.0-2+deb13u1).
Sommaire :
  Mise à niveau de : 0. Installation de : 0Supprimé : 0. Non mis à jour : 0
root@TemplateTerraform:~# _

```

1.4 Création d'un utilisateur standard avec accès sudo

On crée un utilisateur "nemo" non-root avec les droits sudo. Ce compte servira pour les connexions SSH manuelles à la VM, notamment pour l'installation d'outils et les tests réseau.

`adduser nemo`

`usermod -aG sudo nemo`

```
root@TemplateTerraform:~# adduser nemo
Nouveau mot de passe :
Retapez le nouveau mot de passe :
passwd : mot de passe mis à jour avec succès
Modifier les informations associées à un utilisateur pour nemo
Entrer la nouvelle valeur, ou appuyer sur ENTER pour la valeur par défaut
  NOM []: NEMO
  Numéro de chambre []:
  Téléphone professionnel []:
  Téléphone personnel []:
  Autre []:
Is the information correct? [Y/n] Y
root@TemplateTerraform:~# usermod -aG sudo nemo
root@TemplateTerraform:~#
```

groups nemo

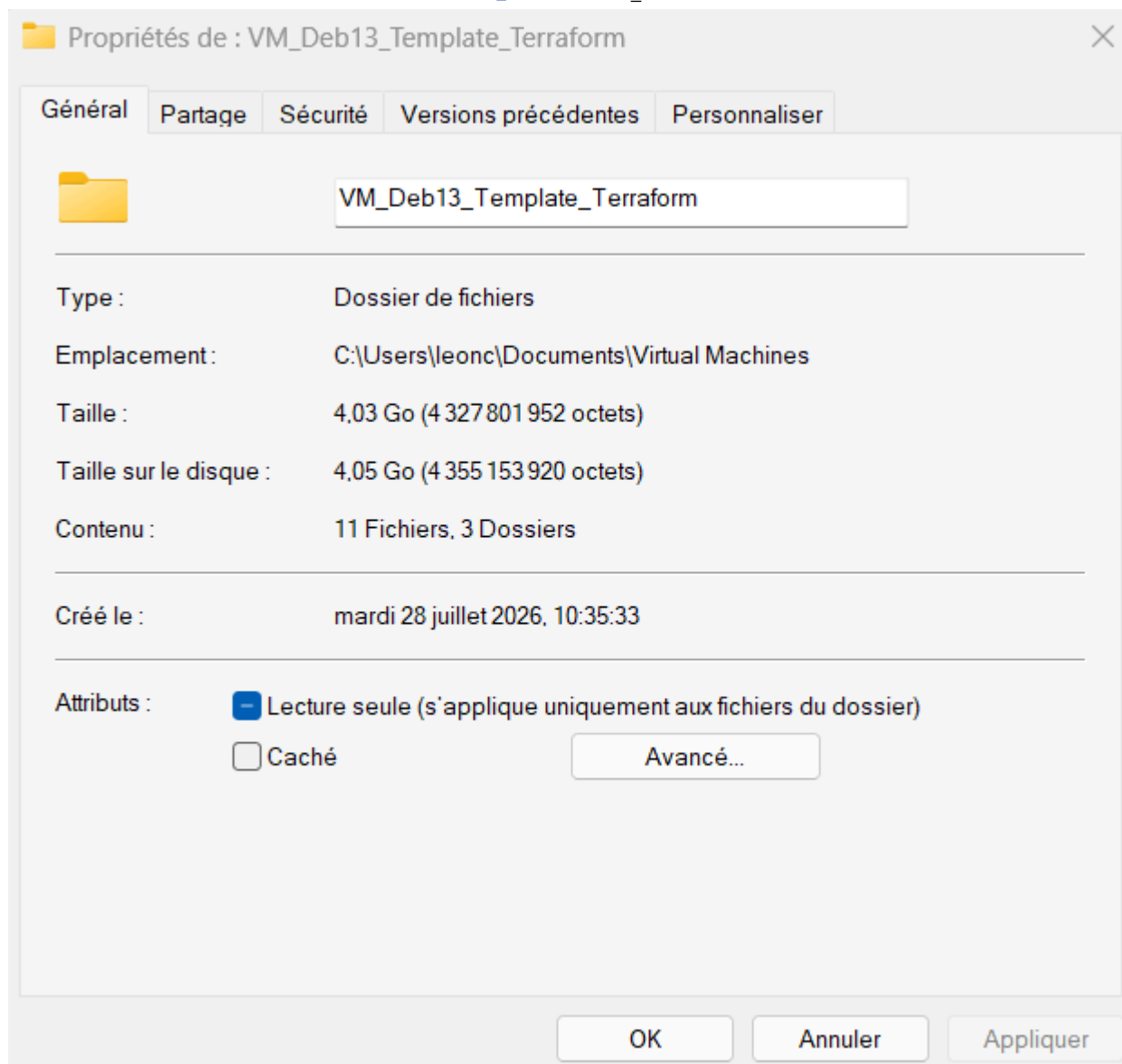
id nemo

```
root@TemplateTerraform:~# groups nemo
nemo : nemo sudo users
root@TemplateTerraform:~# id nemo
uid=1001(nemo) gid=1001(nemo) groupes=1001(nemo),27(sudo),100(users)
root@TemplateTerraform:~# _
```

1.5 Récupération du chemin vers le fichier .vmx

Une fois la VM template prête, on note le chemin complet vers son fichier. Ce chemin est la référence que Terraform utilisera comme source pour cloner de nouvelles VMs.

C:\Users\leonc\Documents\Virtual Machines\VM_Deb13_Template_Terraform



La VM template est prête. Nous allons maintenant installer Terraform et configurer le provider VMware Workstation pour piloter nos VMs.

[Sommaire](#)

2 - Installation de Terraform et configuration du Provider VMware Workstation

Un provider est un plugin permettant à Terraform de communiquer avec une plateforme spécifique, ici VMware Workstation. Pour ce projet nous décidons d'utiliser un provider communautaire disponible sur Github : "terraform-provider-workstation". Ce provider fonctionne avec une API REST type serveur permettant à Terraform de créer, modifier et supprimer des VMs sur VMware Workstation.

2.1 Installation de Terraform CLI

On installe Terraform CLI depuis les dépôts officiels HashiCorp.

2.11 Installation de GPG

```
apt install -y gpg
```

GPG (GNU Privacy Guard) est un outil de chiffrement et de signature numérique. GNU (GNU's Not Unix) est un projet lancé en 1983 par Richard Stallman avec l'objectif de créer un système d'exploitation entièrement libre et open-source, compatible avec Unix. GNU fournit la plupart des outils de base qu'on utilise sous Linux au quotidien : ls, cp, mv, grep, gpg... C'est pourquoi on dit souvent **GNU/Linux** et pas juste "Linux". Le noyau c'est Linux, mais les outils autour c'est GNU.

```
root@TemplateTerraform:~# apt install -y gpg
gpg est déjà la version la plus récente (2.4.7-21+deb13u1+b4).
Sommaire :
  Mise à niveau de : 0. Installation de : 0Supprimé : 0. Non mis à jour : 0
root@TemplateTerraform:~#
```

2.12 Téléchargement de la clé GPG

```
wget -O- https://apt.releases.hashicorp.com/gpg | gpg --dearmor -o
/usr/share/keyrings/hashicorp-archive-keyring.gpg
```

Détail de la commande :

wget : outil qui télécharge des fichiers depuis internet via une URL.

-O- = au lieu de sauvegarder le fichier sur le disque, redirige le contenu directement vers la sortie (le pipe |).

| (pipe) — redirige la sortie d'une commande vers l'entrée d'une autre. Ici, ce que wget télécharge est envoyé directement à gpg.

Le **pipe** | est un opérateur qui connecte deux commandes en chaîne : La **sortie** de la commande 1 devient l'entrée de la commande 2.

gpg --dearmor : convertit la clé GPG du format texte (ASCII) en format binaire compréhensible par apt.

-o /usr/share/keyrings/... : sauvegarde le résultat dans ce fichier. C'est l'emplacement standard où apt cherche les clés de confiance.

```

root@TemplateTerraform:~# wget -O- https://apt.releases.hashicorp.com/gpg |
gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg
--2026-07-28 11:22:05-- https://apt.releases.hashicorp.com/gpg
Résolution de apt.releases.hashicorp.com (apt.releases.hashicorp.com)... 13.22
7.231.4, 13.227.231.45, 13.227.231.121, ...
Connexion à apt.releases.hashicorp.com (apt.releases.hashicorp.com)|13.227.2
31.4|:443... connecté.
requête HTTP transmise, en attente de la réponse... 200 OK
Taille : 3980 (3,9K) [binary/octet-stream]
Sauvegarde en : « STDOUT »

-          100%[=====] 3,89K --.-KB/s  ds 0s

2026-07-28 11:22:05 (202 MB/s) - envoi vers sortie standard [3980/3980]

root@TemplateTerraform:~# |

```

2.13 Ajoût du dépôt HashiCorp dans le fichier

`/etc/apt/sources.list.d/hashicorp.list`

```

echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg]
https://apt.releases.hashicorp.com $(lsb_release -cs) main" | tee
/etc/apt/sources.list.d/hashicorp.list

```

```

root@TemplateTerraform:~# echo "deb [signed-by=/usr/share/keyrings/hashicorp
-archive-keyring.gpg] https://apt.releases.hashicorp.com $(lsb_release -cs)
main" | tee /etc/apt/sources.list.d/hashicorp.list
deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://ap
t.releases.hashicorp.com trixie main
root@TemplateTerraform:~#

```

2.14 Mise à jour de la liste des paquets disponibles à partir du nouveau dépôt HashiCorp

`apt update`

```

root@TemplateTerraform:~# apt update
Atteint : 1 http://deb.debian.org/debian trixie InRelease
Réception de : 2 http://security.debian.org/debian-security trixie-security
InRelease [43,4 kB]
Atteint : 3 http://deb.debian.org/debian trixie-updates InRelease
Réception de : 4 https://apt.releases.hashicorp.com trixie InRelease [12,9 k
B]
Réception de : 5 https://apt.releases.hashicorp.com trixie/main amd64 Packag
es [246 kB]
302 ko réceptionnés en 1s (503 ko/s)
Tous les paquets sont à jour.
root@TemplateTerraform:~# |

```

2.15 Installation de terraform

`apt install terraform`

```

root@TemplateTerraform:~# apt install terraform
Installation de :
  terraform

Installation de dépendances :
  git          liberror-perl  libngtcp2-crypto-gnutls8  patch
  git-man      libnghttp3-9   librtmp1
  libcurl3t64-gnutls  libngtcp2-16  libssh2-1t64

Paquets suggérés :
  git-doc  git-gui  gitweb  git-mediawiki  ed
  git-email  gitk  git-cvs  git-svn  diffutils-doc

Sommaire :
  Mise à niveau de : 0. Installation de : 11Supprimé : 0. Non mis à jour : 0
  Taille du téléchargement : 47,0 MB
  Espace nécessaire : 173 MB / 17,5 GB disponible

Continuer ? [O/n] o
Réception de : 1 http://deb.debian.org/debian trixie/main amd64 libnghttp3-9
amd64 1.8.0-1 [67,7 kB]
Réception de : 2 http://deb.debian.org/debian trixie/main amd64 libngtcp2-16
amd64 1.11.0-1+deb13u1 [132 kB]
Réception de : 3 http://deb.debian.org/debian trixie/main amd64 libngtcp2-cr
ypto-gnutls8 amd64 1.11.0-1+deb13u1 [29,5 kB]
Réception de : 4 http://deb.debian.org/debian trixie/main amd64 librtmp1 amd
64 2.4+20151223.gitfa8646d.1-2+b5 [58,8 kB]
Réception de : 5 http://deb.debian.org/debian trixie/main amd64 libssh2-1t64
amd64 1.11.1-1+deb13u1 [245 kB]
Réception de : 6 https://apt.releases.hashicorp.com trixie/main amd64 terraform
amd64 1.15.8-1 [34,8 MB]
Réception de : 7 http://deb.debian.org/debian trixie/main amd64 libcurl3t64-
gnutls amd64 8.14.1-2+deb13u4 [384 kB]
Réception de : 8 http://deb.debian.org/debian trixie/main amd64 liberror-per
l all 0.17030-1 [26,9 kB]
Réception de : 9 http://deb.debian.org/debian trixie/main amd64 git-man all
1:2.47.3-0+deb13u1 [2 205 kB]
Réception de : 10 http://deb.debian.org/debian trixie/main amd64 git amd64 1
:2.47.3-0+deb13u1 [8 862 kB]
48% [10 git 4 481 kB/8 862 kB 51%] [6 terraform 12,3 MB/34,8 MB 35%]

```

2.16 Vérification de l'installation de terraform

`terraform --version`

```

root@TemplateTerraform:~# terraform --version
Terraform v1.15.8
on linux_amd64
root@TemplateTerraform:~#

```

Pour installer Terraform sur Debian, nous avons suivi 5 étapes. C'est un peu comme télécharger un logiciel sur Windows depuis le site officiel, sauf que sur Linux le processus est plus sécurisé et automatisé. On a d'abord installé **GPG**, l'outil qui va nous permettre de vérifier l'authenticité de ce qu'on télécharge. Ensuite on a téléchargé la **clé GPG de HashiCorp**, qui est la signature numérique officielle de l'éditeur, comme un certificat d'authenticité. On a alors ajouté le **dépôt HashiCorp** à notre liste de sources apt et lancer

un apt update pour recharger la liste des paquets disponibles, et c'est à ce moment que GPG vérifie que le dépôt est bien signé par HashiCorp. Enfin, apt install terraform a téléchargé et installé Terraform depuis ce dépôt vérifié. **HashiCorp** est une entreprise américaine fondée en 2012, spécialisée dans les outils d'infrastructure et de DevOps. Elle est à l'origine de Terraform, mais aussi de Vault (gestion des secrets), Consul (gestion des réseaux) et Vagrant (virtualisation). En 2023, HashiCorp a été rachetée par IBM.

2.2 Lancement du Provider VMware Workstation

Terraform pour parler à VMWare a besoin d'un **provider** : un plugin qui fait le lien entre Terraform et la plateforme cible, ici VMWare. Le provider "terraform-provider-vmworkstation" est un provider **communautaire** (développé par elsudano) qui lance un serveur REST local sur Windows, permettant à Terraform de communiquer avec VMware Workstation Pro via son API sur le port 8697.

VM Debian (terraform apply) → réseau → Windows:8697 (serveur REST) → VMware Workstation Pro

On télécharge le binaire depuis GitHub :

<https://github.com/elsudano/terraform-provider-vmworkstation/releases>

On crée le répertoire attendu par Terraform :

```
PS C:\Users\leonc> New-Item -ItemType Directory -Path "$env:APPDATA\terraform.d\plugins\registry.terraform.io\elsudano\vmworkstation\2.0.1\windows_amd64" -Force
```

Répertoire: C:\Users\leonc\AppData\Roaming\terraform.d\plugins\registry.terraform.io\elsudano\vmworkstation\2.0.1

Mode	LastWriteTime	Length	Name
d-----	29/07/2026 14:21		windows_amd64

On copie le binaire dans ce répertoire :

```
PS C:\Users\leonc> Copy-Item "$env:USERPROFILE\Downloads\terraform-provider-vmworkstation_2.0.1_windows_amd64\terraform-provider-vmworkstation_v2.0.1.exe" "$env:APPDATA\terraform.d\plugins\registry.terraform.io\elsudano\vmworkstation\2.0.1\windows_amd64\"
```

On vérifie que le fichier est bien en place :

```
PS C:\Users\leonc> Get-ChildItem "$env:APPDATA\terraform.d\plugins\registry.terraform.io\elsudano\vmworkstation\2.0.1\windows_amd64\"
```

```
Répertoire: C:\Users\leonc\AppData\Roaming\terraform.d\plugins\registry.terraform.io\elsudano\vmworkstation\2.0.1\windows_amd64
```

Mode		LastWriteTime	Length	Name
----		-----	-----	----
-a----		11/06/2025 17:04	17883648	terraform-provider-vmworkstation_v2.0.1.exe

```
PS C:\Users\leonc>
```

2.3 Lancement du serveur REST

```
PS C:\Users\leonc> & "C:\Program Files (x86)\VMware\VMware Workstation\vmrest.exe"
VMware Workstation REST API
Copyright (C) 2018-2024 VMware Inc.
All Rights Reserved

vmrest 1.3.1 build-24319023
-
Using the VMware Workstation UI while API calls are in progress is not recommended and may yield unexpected results.
-
Serving HTTP on 127.0.0.1:8697
-
Press Ctrl+C to stop.
```

[Sommaire](#)

3 - Premier déploiement de VM Debian 13

Avec Terraform installé et le provider configuré, nous pouvons décrire notre première VM dans un fichier de configuration et la déployer en trois commandes. C'est ici que l'on comprend concrètement l'approche IaC : on décrit l'état souhaité, Terraform se charge de l'atteindre.

3.1 Création de la ressource VM dans main.tf

```

terraform {
  # null_resource est intégré dans Terraform, pas besoin de provider externe
}

# Ressource VM via appel direct à l'API vmrest
resource "null_resource" "debian_lab" {
  provisioner "local-exec" {
    interpreter = ["PowerShell", "-Command"]
    command     = <<-EOT
      $ErrorActionPreference = "Stop"
      $cred = [Convert]::ToBase64String([Text.Encoding]::ASCII.GetBytes("test:Blaise123."))
      $headers = @{
        Authorization = "Basic $cred"
        "Content-Type" = "application/vnd.vmware.vmw.rest-v1+json"
      }
      $body = '{"name": "debian-lab-01", "parentId": "IKTVMKNAQEHHREJG8EQGD7ES895ML1NA"}'
      $result = Invoke-RestMethod -Uri "http://127.0.0.1:8697/api/vms" -Method POST -Headers $headers -Body $body
      Write-Output "VM créée avec ID: $($result.id)"
    EOT
  }
}

```

3.2 Initialisation du projet : terraform init

La commande `terraform init` initialise le répertoire de travail. Avec `null_resource`, aucun provider externe n'est nécessaire. Terraform utilise ici le provider hashicorp/null intégré (v3.3.0) et confirme que le répertoire est prêt.

```

PS C:\Users\leonc\terraform_debian_lab> terraform init
Initializing the backend...

Initializing provider plugins...
- Reusing previous version of hashicorp/null from the dependency lock file
- Using previously-installed hashicorp/null v3.3.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Users\leonc\terraform_debian_lab>

```

3.3 Planification : terraform plan

La commande `terraform plan` calcule et affiche les changements que Terraform va effectuer sans les appliquer.

Terraform affiche un résumé indiquant qu'une ressource va être créée.

```
PS C:\Users\leonc\terraform_debian_lab> terraform plan

Terraform used the selected providers to generate the following execution
plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# null_resource.debian_lab will be created
+ resource "null_resource" "debian_lab" {
  + id          = (known after apply)
  + triggers = {
    + "vm_name"          = "debian-lab-01"
    + "vm_parent_id"     = "IKTVMKNAQEHHREJG8EQGD7ES895ML1NA"
    + "vmrest_password"  = (sensitive value)
    + "vmrest_url"       = "http://127.0.0.1:8697"
    + "vmrest_user"      = "test"
  }
}

Plan: 1 to add, 0 to change, 0 to destroy.
```

Note : ce plan est issu de la section 4 (avec variables). En section 3, un `terraform apply` direct avait été exécuté sans `terraform plan` préalable

3.4 Déploiement : terraform apply

La commande `terraform apply` applique réellement les changements. Terraform demande une confirmation avant de procéder puis crée la VM dans VMware Workstation.

Après confirmation, Terraform crée la VM et génère le fichier `terraform.tfstate` qui enregistre l'état de l'infrastructure gérée.

```

PS C:\Users\leonc\terraform_debian_lab> terraform apply

Terraform used the selected providers to generate the following execution
plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# null_resource.debian_lab will be created
+ resource "null_resource" "debian_lab" {
  + id = (known after apply)
}

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

null_resource.debian_lab: Creating...
null_resource.debian_lab: Provisioning with 'local-exec'...
null_resource.debian_lab (local-exec): Executing: ["PowerShell" "-Command" "
$errorActionPreference = \"Stop\"\\r\\n$cred = [Convert]::ToBase64String([Text
.Encoding]::ASCII.GetBytes(\"test:Blaise123.\"))\\r\\n$headers = @{\\r\\n Autho
rization = \"Basic $cred\"\\r\\n \"Content-Type\" = \"application/vnd.vmware
.vmw.rest-v1+json\"\\r\\n}\\r\\n$body = '{\\\"name\\\": \\\"debian-lab-01\\\", \\\"parentI
d\\\": \\\"IKTVMKNAQEHREJG8EQGD7ES895ML1NA\\\"}'\\r\\n$result = Invoke-RestMethod -
Uri \"http://127.0.0.1:8697/api/vms\" -Method POST -Headers $headers -Body $
body\\r\\nWrite-Output \"VM créée avec ID: $($result.id)\"\\r\\n"]
null_resource.debian_lab (local-exec): VM créée avec ID: UM5SRRSG12GK39R9CPF
ISKONE10V501M
null_resource.debian_lab: Creation complete after 6s [id=8982572095798858387
]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
PS C:\Users\leonc\terraform_debian_lab>

```

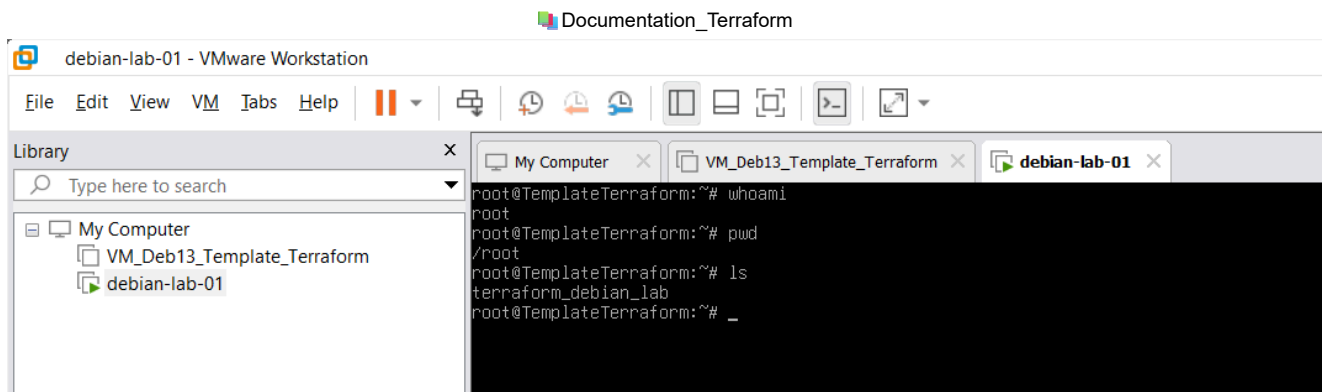
3.5 Vérification du déploiement

On vérifie dans VMware Workstation que la VM debian-lab-01 est bien créée et démarrée.
On peut également consulter l'état Terraform avec la commande `terraform show`.

```

PS C:\Users\leonc\terraform_debian_lab> terraform show
# null_resource.debian_lab:
resource "null_resource" "debian_lab" {
  id = "8982572095798858387"
}
PS C:\Users\leonc\terraform_debian_lab>

```



La première VM est déployée par Terraform. Nous allons maintenant améliorer la configuration en introduisant des variables et en explorant la gestion du cycle de vie.

[Sommaire](#)

4 - Variables et gestion du cycle de vie

Jusqu'ici les valeurs (nom de la VM, ID du remplate, mdp etc...) étaient écrites en dur dans `main.tf`. Un fichier `variables.tf` sert à déclarer les variables que la configuration va utiliser. On y renseigne :

- le nom de la variable
- le type (string, number, bool etc...)
- une description (pour la documentation)
- une valeur par défaut (éventuellement)

Pour les données sensibles on utilise le paramètre `sensitive=true` ce qui permet de masquer les valeurs dans les logs

4.1 Création de `variables.tf`

On crée un fichier `variables.tf` qui déclare toutes les variables utilisées par la configuration, avec leur type et une description avec la commande `notepad variables.tf` dans le dossier `C:\Users\leonc\terraform_debian_lab`

```

variables.tf
1 variable "vm_name" {
2   description = "Nom de la VM à créer dans VMware"
3   type        = string
4   default     = "debian-lab-01"
5 }
6
7 variable "vm_parent_id" {
8   description = "ID de la VM template source (obtenu via GET /api/vms)"
9   type        = string
10 }
11
12 variable "vmrest_url" {
13   description = "URL de l'API vmrest"
14   type        = string
15   default     = "http://127.0.0.1:8697"
16 }
17
18 variable "vmrest_user" {
19   description = "Nom d'utilisateur vmrest"
20   type        = string
21 }
22
23 variable "vmrest_password" {
24   description = "Mot de passe vmrest"
25   type        = string
26   sensitive   = true
27 }

```

4.2 Création de terraform.tfvars

Edition du fichier de configuration terraform.tfvars avec la commande **notepad** **terraform.tfvars** qui contient les valeurs réelles des variables. Ce fichier ne doit pas être publié sur Git si il contient des données sensibles.

```

variables.tf  terraform.tfvars
1 vm_name      = "debian-lab-01"
2 vm_parent_id = "IKTVMKNAQEHHREJG8EQGD7ES895ML1NA"
3 vmrest_url   = "http://127.0.0.1:8697"
4 vmrest_user  = "test"
5 vmrest_password = "Blaise123."

```

4.3 Mise à jour de main.tf pour utiliser les variables

Dans **main.tf** tout est codé en dur et donc lisible. L'objectif est de remplacer chaque valeur par **\${var.nom_variable}** (syntaxe terraform pour injecter une variable dans une chaîne de texte). On ajoute également un bloc **triggers** dans le **null_resource** afin de stocker les valeurs des variables dans le state Terraform ce qui est indispensable pour que la commande terraform destroy puisse être exécuté au moment de la destruction. C'est grâce aux triggers que Terraform sait quand créer la ressource : si une valeur du trigger change,

Terraform détruit et recrée la VM. On édite donc le fichier de configuration **main.tf** avec la commande **notepad main.tf**

```

1 terraform {
2 }
3
4 resource "null_resource" "debian_lab" {
5
6   triggers = {
7     vm_name      = var.vm_name
8     vm_parent_id = var.vm_parent_id
9     vmrest_url   = var.vmrest_url
10    vmrest_user  = var.vmrest_user
11    vmrest_password = var.vmrest_password
12  }
13
14  provisioner "local-exec" {
15    interpreter = ["PowerShell", "-Command"]
16    command     = <<-EOT
17      $ErrorActionPreference = "Stop"
18      $cred = [Convert]::ToBase64String([Text.Encoding]::ASCII.GetBytes("${var.vmrest_user}:${var.vmrest_password}"))
19      $headers = @{
20        Authorization = "Basic $cred"
21        "Content-Type" = "application/vnd.vmware.vmw.rest-v1+json"
22      }
23      $body = '{"name": "${var.vm_name}", "parentId": "${var.vm_parent_id}"}'
24      $result = Invoke-RestMethod -Uri "${var.vmrest_url}/api/vms" -Method POST -Headers $headers -Body $body
25      Write-Output "VM créée avec ID: ${result.id}"
26    EOT
27  }
28 }

```

4.4 Gestion des modifications

Vérifier que VMREST tourne bien avec la commande **& "C:\Program Files (x86)\VMware\VMware Workstation\vmrest.exe"**

```

PS C:\Users\leonc> & "C:\Program Files (x86)\VMware\VMware Workstation\vmrest.exe"
VMware Workstation REST API
Copyright (C) 2018-2024 VMware Inc.
All Rights Reserved

vmrest 1.3.1 build-24319023
-
Using the VMware Workstation UI while API calls are in progress is not recommended and may yield unexpected results.
-
Serving HTTP on 127.0.0.1:8697
-
Press Ctrl+C to stop.

```

```

PS C:\Users\leonc\terraform_debian_lab> terraform show
# null_resource.debian_lab:
resource "null_resource" "debian_lab" {
  id = "8982572095798858387"
}
PS C:\Users\leonc\terraform_debian_lab>

```

Avant d'appliquer destruction de la VM existante debian-lab-01 qui a été créée (sinon l'API vmrest va retourner une erreur 409 "VM already exists") avec la commande **terraform destroy**


```

PS C:\Users\leonc\terraform_debian_lab> terraform destroy
var.vm_parent_id
  ID de la VM template source (obtenu via GET /api/vms)

  Enter a value: yes

var.vmrest_password
  Mot de passe vmrest

  Enter a value:

var.vmrest_user
  Nom d'utilisateur vmrest

  Enter a value: test

null_resource.debian_lab: Refreshing state... [id=8982572095798858387]

Terraform used the selected providers to generate the following execution
plan. Resource actions are indicated with the following symbols:
  - destroy

Terraform will perform the following actions:

# null_resource.debian_lab will be destroyed
- resource "null_resource" "debian_lab" {
  - id = "8982572095798858387" -> null
}

Plan: 0 to add, 0 to change, 1 to destroy.

Do you really want to destroy all resources?
  Terraform will destroy all your managed infrastructure, as shown above.
  There is no undo. Only 'yes' will be accepted to confirm.

  Enter a value: yes

null_resource.debian_lab: Destroying... [id=8982572095798858387]
null_resource.debian_lab: Destruction complete after 0s

Destroy complete! Resources: 1 destroyed.
PS C:\Users\leonc\terraform_debian_lab>

```

Vérification avec la commande `terraform show`

```

PS C:\Users\leonc\terraform_debian_lab> terraform show
The state file is empty. No resources are represented.
PS C:\Users\leonc\terraform_debian_lab>

```

Suppression du dossier VM restant sur le disque avec la commande `Remove-Item -Recurse -Force "C:\Users\leonc\Documents\Virtual Machines\debian-lab-01"`

Afficher les modifications détectées avec la commande `terraform plan`

```
PS C:\Users\leonc\terraform_debian_lab> terraform plan

Terraform used the selected providers to generate the following execution
plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# null_resource.debian_lab will be created
+ resource "null_resource" "debian_lab" {
  + id          = (known after apply)
  + triggers = {
    + "vm_name"          = "debian-lab-01"
    + "vm_parent_id"     = "IKTVMKNAQEHHREJG8EQGD7ES895ML1NA"
    + "vmrest_password"  = (sensitive value)
    + "vmrest_url"       = "http://127.0.0.1:8697"
    + "vmrest_user"      = "test"
  }
}

Plan: 1 to add, 0 to change, 0 to destroy.
```

Application des modifications avec la commande `terraform apply`

```
PS C:\Users\leonc\terraform_debian_lab> terraform apply
```

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:

+ create

Terraform will perform the following actions:

```
# null_resource.debian_lab will be created
+ resource "null_resource" "debian_lab" {
  + id          = (known after apply)
  + triggers = {
    + "vm_name"          = "debian-lab-01"
    + "vm_parent_id"     = "IKTVMKNAQEHHREJG8EQGD7ES895ML1NA"
    + "vmrest_password"  = (sensitive value)
    + "vmrest_url"       = "http://127.0.0.1:8697"
    + "vmrest_user"      = "test"
  }
}
```

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?

Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

```
null_resource.debian_lab: Creating...
null_resource.debian_lab: Provisioning with 'local-exec'...
null_resource.debian_lab (local-exec): (output suppressed due to sensitive v
alue in config)
null_resource.debian_lab (local-exec): (output suppressed due to sensitive v
alue in config)
null_resource.debian_lab: Creation complete after 10s [id=653012353945919970
]
```

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

```
PS C:\Users\leonc\terraform_debian_lab> |
```

Vérification avec la commande `terraform show`

```
PS C:\Users\leonc\terraform_debian_lab> terraform show
# null_resource.debian_lab:
resource "null_resource" "debian_lab" {
  id          = "653012353945919970"
  triggers = {
    "vm_name"          = "debian-lab-01"
    "vm_parent_id"     = "IKTVMKNAQEHHREJG8EQGD7ES895ML1NA"
    "vmrest_password"  = (sensitive value)
    "vmrest_url"       = "http://127.0.0.1:8697"
    "vmrest_user"      = "test"
  }
}
PS C:\Users\leonc\terraform_debian_lab>
```

Destruction de la VM avec la commande `terraform destroy`

```

PS C:\Users\leonc\terraform_debian_lab> terraform destroy
null_resource.debian_lab: Refreshing state... [id=653012353945919970]

Terraform used the selected providers to generate the following execution
plan. Resource actions are indicated with the following symbols:
  - destroy

Terraform will perform the following actions:

# null_resource.debian_lab will be destroyed
- resource "null_resource" "debian_lab" {
  - id          = "653012353945919970" -> null
  - triggers = {
    - "vm_name"          = "debian-lab-01"
    - "vm_parent_id"     = "IKTVMKNAQEHHREJG8EQGD7ES895ML1NA"
    - "vmrest_password"  = (sensitive value)
    - "vmrest_url"       = "http://127.0.0.1:8697"
    - "vmrest_user"      = "test"
  } -> null
}

Plan: 0 to add, 0 to change, 1 to destroy.

Do you really want to destroy all resources?
Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

null_resource.debian_lab: Destroying... [id=653012353945919970]
null_resource.debian_lab: Destruction complete after 0s

Destroy complete! Resources: 1 destroyed.
PS C:\Users\leonc\terraform_debian_lab>

```

Vérification avec la commande `terraform show`

```

PS C:\Users\leonc\terraform_debian_lab> terraform show
The state file is empty. No resources are represented.
PS C:\Users\leonc\terraform_debian_lab> |

```

[Sommaire](#)

5 - Laboratoire de Cybersécurité déploiement de deux VMs

L'objectif final du projet est de déployer un mini-laboratoire cybersécurité composé de deux VMs sur le même réseau virtuel : une VM "attaquante" (kali-like-vm) équipée d'outils de scan réseau, et une VM "victime" (victim-vm) exposant un service accessible sur le réseau.

Terraform orchestre le déploiement et la configuration des deux machines simultanément via des provisioners.

5.1 Architecture du laboratoire

5.2 Déclaration de la VM attaquante dans main.tf

Edition du fichier de configuration main.tf avec la commande `notepad main.tf`

```

1 terraform {
2 }
3
4 # VM Attaquante
5 resource "null_resource" "kali_like" {
6   triggers = {
7     vm_name      = "kali-like-vm"
8     vm_parent_id = var.vm_parent_id
9     vmrest_url   = var.vmrest_url
10    vmrest_user  = var.vmrest_user
11    vmrest_password = var.vmrest_password
12  }
13
14  provisioner "local-exec" {
15    interpreter = ["PowerShell", "-Command"]
16    command     = <<-EOT
17      $ErrorActionPreference = "Stop"
18      $cred = [Convert]::ToBase64String([Text.Encoding]::ASCII.GetBytes("${var.vmrest_user}:${var.vmrest_password}"))
19      $headers = @{
20        Authorization = "Basic $cred"
21        "Content-Type" = "application/vnd.vmware.vmw.rest-v1+json"
22      }
23      $body = '{"name": "kali-like-vm", "parentId": "${var.vm_parent_id}"}'
24      $result = Invoke-RestMethod -Uri "${var.vmrest_url}/api/vms" -Method POST -Headers $headers -Body $body
25      Write-Output "VM attaquante créée avec ID: $($result.id)"
26    EOT
27  }
28 }
29

```

5.3 Déclaration de la VM victime dans main.tf

Edition du fichier de configuration main.tf avec la commande `notepad main.tf`

```

29
30 # VM Victime
31 resource "null_resource" "victim" {
32   triggers = {
33     vm_name      = "victim-vm"
34     vm_parent_id = var.vm_parent_id
35     vmrest_url   = var.vmrest_url
36     vmrest_user  = var.vmrest_user
37     vmrest_password = var.vmrest_password
38   }
39
40   provisioner "local-exec" {
41     interpreter = ["PowerShell", "-Command"]
42     command     = <<-EOT
43       $ErrorActionPreference = "Stop"
44       $cred = [Convert]::ToBase64String([Text.Encoding]::ASCII.GetBytes("${var.vmrest_user}:${var.vmrest_password}"))
45       $headers = @{
46         Authorization = "Basic $cred"
47         "Content-Type" = "application/vnd.vmware.vmw.rest-v1+json"
48       }
49       $body = '{"name": "victim-vm", "parentId": "${var.vm_parent_id}"}'
50       $result = Invoke-RestMethod -Uri "${var.vmrest_url}/api/vms" -Method POST -Headers $headers -Body $body
51       Write-Output "VM victime créée avec ID: $($result.id)"
52     EOT
53   }
54 }

```

5.4 Déploiement simultané des deux VMs

Vérification des modifications du fichier main.tf avec la commande `terraform plan`

```
PS C:\Users\leonc\terraform_debian_lab> terraform plan
```

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:

+ create

Terraform will perform the following actions:

```
# null_resource.kali_like will be created
+ resource "null_resource" "kali_like" {
+   id          = (known after apply)
+   triggers = {
+     "vm_name"          = "kali-like-vm"
+     "vm_parent_id"     = "IKTVMKNAQEHREJG8EQGD7ES895ML1NA"
+     "vmrest_password"  = (sensitive value)
+     "vmrest_url"       = "http://127.0.0.1:8697"
+     "vmrest_user"      = "test"
+   }
+ }

# null_resource.victim will be created
+ resource "null_resource" "victim" {
+   id          = (known after apply)
+   triggers = {
+     "vm_name"          = "victim-vm"
+     "vm_parent_id"     = "IKTVMKNAQEHREJG8EQGD7ES895ML1NA"
+     "vmrest_password"  = (sensitive value)
+     "vmrest_url"       = "http://127.0.0.1:8697"
+     "vmrest_user"      = "test"
+   }
+ }
```

Plan: 2 to add, 0 to change, 0 to destroy.

Note: You didn't use the `-out` option to save this plan, so Terraform can't guarantee to take exactly these actions if you run `"terraform apply"` now.

```
PS C:\Users\leonc\terraform_debian_lab>
```

Application des modifications avec la commande `terraform apply`

```
PS C:\Users\leonc\terraform_debian_lab> terraform apply
```

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:

+ create

Terraform will perform the following actions:

```
# null_resource.kali_like will be created
+ resource "null_resource" "kali_like" {
  + id          = (known after apply)
  + triggers = {
    + "vm_name"          = "kali-like-vm"
    + "vm_parent_id"     = "IKTVMKNAQEHHREJG8EQGD7ES895ML1NA"
    + "vmrest_password" = (sensitive value)
    + "vmrest_url"      = "http://127.0.0.1:8697"
    + "vmrest_user"     = "test"
  }
}

# null_resource.victim will be created
+ resource "null_resource" "victim" {
  + id          = (known after apply)
  + triggers = {
    + "vm_name"          = "victim-vm"
    + "vm_parent_id"     = "IKTVMKNAQEHHREJG8EQGD7ES895ML1NA"
    + "vmrest_password" = (sensitive value)
    + "vmrest_url"      = "http://127.0.0.1:8697"
    + "vmrest_user"     = "test"
  }
}
```

Plan: 2 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?

Terraform will perform the actions described above.

Only 'yes' will be accepted to approve.

Enter a value: yes

```
null_resource.kali_like: Creating...
null_resource.kali_like: Provisioning with 'local-exec'...
null_resource.kali_like (local-exec): (output suppressed due to sensitive value in config)
null_resource.kali_like (local-exec): (output suppressed due to sensitive value in config)
null_resource.kali_like: Creation complete after 4s [id=3113304537758926692]
null_resource.victim: Creating...
null_resource.victim: Provisioning with 'local-exec'...
null_resource.victim (local-exec): (output suppressed due to sensitive value in config)
null_resource.victim (local-exec): (output suppressed due to sensitive value in config)
null_resource.victim: Creation complete after 4s [id=6288942191835585317]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
PS C:\Users\leonc\terraform_debian_lab>
```

Vérification de la présence des deux VMs dans le dossier Virtual Machines avec la commande `ls "C:\Users\leonc\Documents\Virtual Machines"`

```
PS C:\Users\leonc> ls "C:\Users\leonc\Documents\Virtual Machines\"

Répertoire : C:\Users\leonc\Documents\Virtual Machines

Mode                LastWriteTime         Length Name
----                -
d-----          30/07/2026   11:41             kali-like-vm
d-----          30/07/2026   11:42             victim-vm
d-----          30/07/2026   11:42             VM_Deb13_Template_Terrafo
rm

PS C:\Users\leonc>
```

5.5 Vérification de la connectivité et du scénario

Installation de nmap sur la vm kali-like-vm avec la commande `apt install -y nmap masscan`

```
root@TemplateTerraform:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:84:c1:35 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    altname enx000c2984c135
    inet 192.168.163.144/24 brd 192.168.163.255 scope global dynamic noprefixroute ens33
        valid_lft 1126sec preferred_lft 901sec
    inet6 fe80::7054:de42:4f57:c3c5/64 scope link
        valid_lft forever preferred_lft forever
root@TemplateTerraform:~#
```



```

Taille du téléchargement : 7 069 kB
Espace nécessaire : 29,6 MB / 17,3 GB disponible

Réception de : 1 http://deb.debian.org/debian trixie/main amd64 libblas3 amd64 3.12.1-6 [160 kB]
Réception de : 2 http://deb.debian.org/debian trixie/main amd64 liblinear4 amd64 2.3.0+dfsg-5+b2 [41,7 kB]
Réception de : 3 http://deb.debian.org/debian trixie/main amd64 liblua5.4-0 amd64 5.4.7-1+b2 [147 kB]
Réception de : 4 http://deb.debian.org/debian trixie/main amd64 libpcap0.8t64 amd64 1.10.5-2 [169 kB]
Réception de : 5 http://deb.debian.org/debian trixie/main amd64 masscan amd64 2:1.3.2+ds1-2 [230 kB]
Réception de : 6 http://deb.debian.org/debian trixie/main amd64 nmap-common all 7.95+dfsg-3 [4 392 kB]
Réception de : 7 http://deb.debian.org/debian trixie/main amd64 nmap amd64 7.95+dfsg-3 [1 931 kB]
7 069 ko réceptionnés en 2s (3 555 ko/s)
Sélection du paquet libblas3:amd64 précédemment désélectionné.
(Lecture de la base de données... 37593 fichiers et répertoires déjà installés.)
Préparation du dépaquetage de .../0-libblas3_3.12.1-6_amd64.deb ...
Dépaquetage de libblas3:amd64 (3.12.1-6) ...
Sélection du paquet liblinear4:amd64 précédemment désélectionné.
Préparation du dépaquetage de .../1-liblinear4_2.3.0+dfsg-5+b2_amd64.deb ...
Dépaquetage de liblinear4:amd64 (2.3.0+dfsg-5+b2) ...
Sélection du paquet liblua5.4-0:amd64 précédemment désélectionné.
Préparation du dépaquetage de .../2-liblua5.4-0_5.4.7-1+b2_amd64.deb ...
Dépaquetage de liblua5.4-0:amd64 (5.4.7-1+b2) ...
Sélection du paquet libpcap0.8t64:amd64 précédemment désélectionné.
Préparation du dépaquetage de .../3-libpcap0.8t64_1.10.5-2_amd64.deb ...
Dépaquetage de libpcap0.8t64:amd64 (1.10.5-2) ...
Sélection du paquet masscan précédemment désélectionné.
Préparation du dépaquetage de .../4-masscan_2%3a1.3.2+ds1-2_amd64.deb ...
Dépaquetage de masscan (2:1.3.2+ds1-2) ...
Sélection du paquet nmap-common précédemment désélectionné.
Préparation du dépaquetage de .../5-nmap-common_7.95+dfsg-3_all.deb ...
Dépaquetage de nmap-common (7.95+dfsg-3) ...
Sélection du paquet nmap précédemment désélectionné.
Préparation du dépaquetage de .../6-nmap_7.95+dfsg-3_amd64.deb ...
Dépaquetage de nmap (7.95+dfsg-3) ...
Paramétrage de libblas3:amd64 (3.12.1-6) ...
update-alternatives: utilisation de « /usr/lib/x86_64-linux-gnu/blas/libblas.so.3 » pour fournir « /usr/lib/x86_64-linux-gnu/libblas.so.3 » en mode automatique
Paramétrage de nmap-common (7.95+dfsg-3) ...
Paramétrage de liblua5.4-0:amd64 (5.4.7-1+b2) ...
Paramétrage de libpcap0.8t64:amd64 (1.10.5-2) ...
Paramétrage de liblinear4:amd64 (2.3.0+dfsg-5+b2) ...
Paramétrage de masscan (2:1.3.2+ds1-2) ...
Paramétrage de nmap (7.95+dfsg-3) ...
Traitement des actions différées (« triggers ») pour man-db (2.13.1-1) ...
Traitement des actions différées (« triggers ») pour libc-bin (2.41-12+deb13u3) ...
root@TemplateTerraform:~# apt install -y nmap masscan
nmap est déjà la version la plus récente (7.95+dfsg-3).
masscan est déjà la version la plus récente (2:1.3.2+ds1-2).
Sommaire :
  Mise à niveau de : 0. Installation de : 0Supprimé : 0. Non mis à jour : 0
root@TemplateTerraform:~#

```

Installation d'un serveur python web sur la vm victim-vm avec la commande **nohup python3 -m http.server 8080 &**

```

root@TemplateTerraform:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:c4:68:90 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    altname enx000c29c46890
    inet 192.168.163.145/24 brd 192.168.163.255 scope global dynamic noprefixroute ens33
        valid_lft 1122sec preferred_lft 897sec
    inet6 fe80::5316:dd82:30b5:6400/64 scope link
        valid_lft forever preferred_lft forever
root@TemplateTerraform:~#

```

```

root@TemplateTerraform:~# nohup python3 -m http.server 8080 &
[1] 1193
root@TemplateTerraform:~# nohup: les entrées sont ignorées et la sortie est ajoutée à 'nohup.out'
^C
root@TemplateTerraform:~# ss -tlnp | grep 8080
LISTEN 0      5            0.0.0.0:8080    0.0.0.0:*      users:(("python3",pid=1193,fd=3))
root@TemplateTerraform:~#

```

Lancement d'un scan avec nmap depuis la vm kali dsur la vm victim avec la commande
nmap -p 8080 192.168.163.145

```
root@TemplateTerraform:~# nmap -p 8080 192.168.163.145
Starting Nmap 7.95 ( https://nmap.org ) at 2026-07-30 11:58 CEST
Nmap scan report for 192.168.163.145
Host is up (0.00075s latency).

PORT      STATE SERVICE
8080/tcp  open  http-proxy
MAC Address: 00:0C:29:C4:68:90 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 0.20 seconds
root@TemplateTerraform:~#
```

nmap détecte que le port 8080 est ouvert sur la victim-vm et identifie le service comme http-proxy

Lancement d'un scan complet avec la commande **nmap -sV 192.168.163.145**

```
root@TemplateTerraform:~# nmap -sV 192.168.163.145
Starting Nmap 7.95 ( https://nmap.org ) at 2026-07-30 14:13 CEST
Nmap scan report for 192.168.163.145
Host is up (0.00013s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 10.0p2 Debian 7+deb13u4 (protocol 2.0)
8080/tcp   open  http      SimpleHTTPServer 0.6 (Python 3.13.5)
MAC Address: 00:0C:29:C4:68:90 (VMware)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 19.40 seconds
root@TemplateTerraform:~#
```

Lancement d'un scan rapide avec masscan avec la commande **masscan 192.168.163.145 -p 8080**

```
root@TemplateTerraform:~# masscan 192.168.163.145 -p 8080
Starting masscan 1.3.2 (http://bit.ly/14GZzcT) at 2026-07-30 12:16:12 GMT
Initiating SYN Stealth Scan
Scanning 1 hosts [1 port/host]
Discovered open port 8080/tcp on 192.168.163.145
rate: 0.00-kpps, 100.00% done, waiting -12-secs, found=1
```

nmap -sV a détecté :

- Port 22 → **OpenSSH 10.0p2 Debian** (le SSH qu'on a installé sur le template)
- Port 8080 → **SimpleHTTPServer 0.6 (Python 3.13.5)** — nmap identifie exactement le service Python
- OS détecté : **Linux, Debian**

[Sommaire](#)

6 - Conclusion et compétences développées sur le projet

Ce projet nous a permis de parcourir l'ensemble du cycle de vie d'une infrastructure pilotée par Terraform, de la préparation de l'environnement VMware jusqu'au déploiement d'un laboratoire cybersécurité fonctionnel. Nous avons appris à décrire une infrastructure sous forme de code, à la versionner, la modifier et la détruire de manière reproductible et contrôlée.

Ce projet m'a permis de mettre en pratique plusieurs compétences concrètes :

- **Infrastructure as Code** : écriture de fichiers `.tf` en HCL, gestion du cycle de vie (init / plan / apply / destroy), utilisation de variables et du fichier `terraform.tfvars`
- **Virtualisation** : clonage de VMs depuis un template Debian 13 via l'API REST de VMware Workstation (`vmrest.exe`)
- **Cybersécurité réseau** : déploiement d'un lab à deux VMs (attaquante / victime), scan de ports avec `nmap` et `masscan` , exposition d'un service HTTP avec Python

[Sommaire](#)
